

1. "There is no computer art so fast that we do not need to spend time to reduce time complexity of an algorithm." - Give argument for or against this statement. 2
 2. Mention two real life areas where algorithms are used. 1
 3. Show that $O(n^3) + O(n^2) = O(n^3)$. 2
 4. If $f(n) = O(g(n))$ iff for some positive constants c_1, c_2 and $n_0, c_1g(n) \leq f(n) \leq c_2g(n)$ for all $n \geq n_0$. 3
- Assume $f(n) = 2n^3 + 10n^2 + 20n$ and $g(n) = n^3$. Show $f(n) = O(g(n))$ by plotting the functions $f(n)$ and $g(n)$ in a figure and indicating c_1, c_2 and n_0 . An approximate figure is sufficient.
5. Consider an array $A = \{5, 2, 3, 4, 6, 1, 7\}$ for merge sort.
 - i) How many Merge-sort call will be there when the initial call is on array A ? 1
 - ii) How many Merge operation will be there? 1

Consider the following function $\text{Fun}(A)$. Assume that A is a sorted sequence of numbers.

$\text{Fun}(A, p, r)$

if ($p < r$)

Let q is a randomly selected index in the array $A[p..r]$

Call $\text{Fun}(A, p, q-1)$

Print the numbers in the array $A[q..r]$

- i) Express the worst case running time of the function Fun using a recurrence. 1
 - ii) Express the best case running time of the function Fun using a recurrence. 1
 - iii) Solve the recurrence of worst case and express it using an asymptotic notation. 2
- Show the operation of the PARTITION of the quick sort algorithm on the array 3
- $A = \{8, 1, 7, 3, 2, 4, 6, 5\}$
- Illustrate the operation of Build-Max-Heap($A, 1$) on the following array 3
- $A = \{5, 13, 9, 5, 12, 8, 7, 4, 0, 6, 2, 1\}$

What do you understand by optimization problems? Which of the following problems are optimization problems? Give justification in support of your answer. 2

- i) Finding an empty class room from the list of class rooms
- ii) Finding a way to best utilize the class rooms of the department

What is overlapping subproblem property? Show that following function for generating Fibonacci sequence has this property. 2

$\text{Fibonacci}(n)$

if ($n \leq 1$) return n ;

else return $\text{Fibonacci}(n-1) + \text{Fibonacci}(n-2)$;

- i) How can you solve the problem of finding Fibonacci number shown above using dynamic programming? 2

- 4.
- a) Illustrate the operation of **Build-Max-Heap(A)** on the following array. Draw the tree, as it is after each **Max-Heapify** call.
 $A = \langle 5, 6, 2, 5, 8, 3, 6, 7, 1, 4, 9 \rangle$
- b) Mention the name of the operations supported by a priority queue?
- c) Explain how $f(x) = 4x^3 - 5x + 3$ is not $O(x^2)$.
- d) Mention the sequence of four steps to develop a dynamic programming algorithm.
- e) Using suitable example show that a recursive algorithm of matrix-chain-multiplication problem calls the subproblem repeatedly.

(1)

ID: C211202

a Time complexity is a measure of the amount of time an algorithm takes to run as a function of the input size. It helps us understand how the execution time of an algorithm grows with larger inputs. It's usually expressed using big O notation, such as $O(1)$, $O(n)$, $O(n^2)$ etc. We don't need to spend time to reduce the time complexity of an algorithm when the input size is small & the algorithm's performance is already acceptable for our needs.

(b) Algorithms are used in numerous real-life areas. Two examples—

① Search Engines → Search engines like Google use complex algorithms to quickly & accurately retrieve relevant web pages in response to user queries.

These algorithms consider factors like keyword relevance, page ranking & user behaviour to provide the most relevant results.

② Financial analysis → They use algorithms like stock trading, risk assessment, fraud detection.

Date: _____

2 To show $O(n^3) + O(n^3) = O(n^3)$, we need to prove that the sum of these two complexities can be bounded by a constant multiple of $O(n^3)$ as the input size grows.

$$O(n^3) + O(n^3) \leq C * n^3$$

This satisfies the definition of $O(n^3)$, as it's an upper bound that grows no faster than n^3 .

(2)

a

(i) $T(n) = T(n-1) + T(0) + \cancel{T(0)} + O(n)$

here $T(n)$ represents the worst case running time on an input size of n .

In the worst case, the randomly selected index q could be the smallest / largest value betⁿ p & r , which would lead to

unbalanced partitioning.

(ii) In the best case, the randomly selected index q would be close to the middle of the subarray, leading to a more balanced partitioning.

$$T(n) = 2^k T(n/2) + O(n)$$

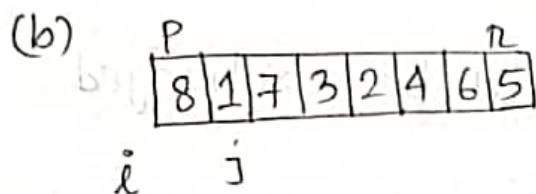
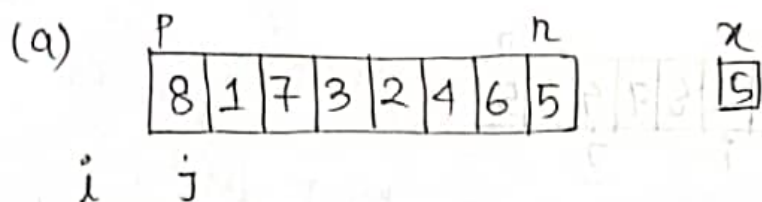
(iii) The recurrence for the worst case is not a simple recurrence that can be directly solved using standard methods like Master Theorem. However we can use a recursion tree method to estimate its behaviour.

Worst case $\rightarrow T(n) = O(n^2)$

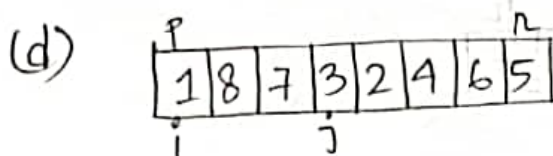
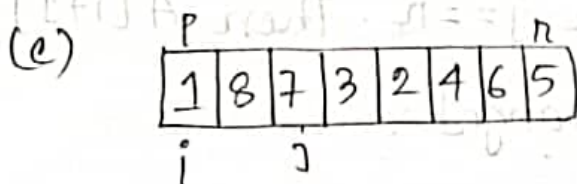
2b. Show the operation of the PARTITION of the Quick sort algorithm on the array.

$$A = \langle 8, 1, 7, 3, 2, 4, 6, 5 \rangle$$

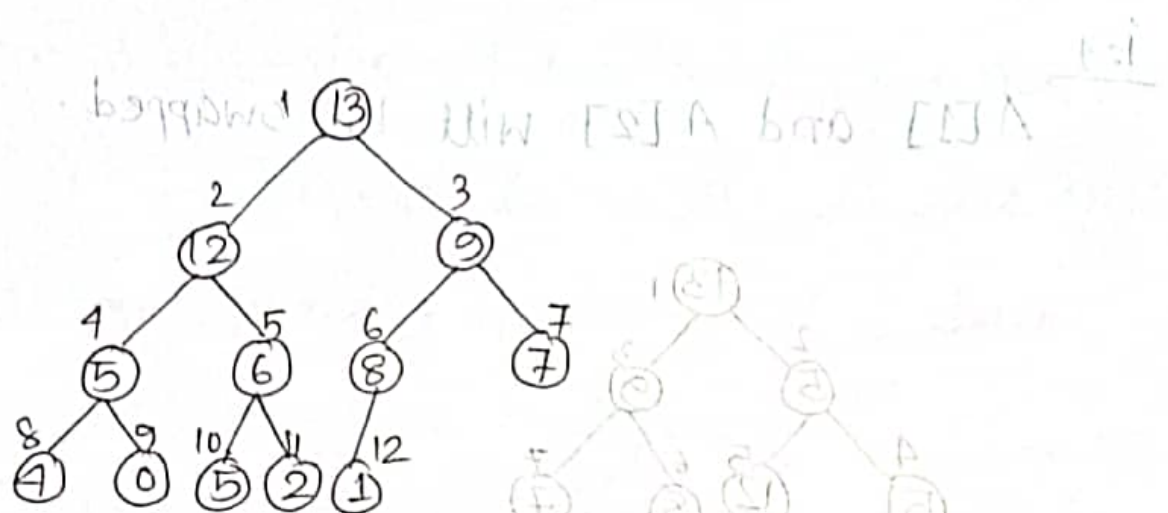
Ans:



since $A[j] \leq x$, $A[i+1]$ and $A[j]$ will be exchanged and i will be incremented by 1.



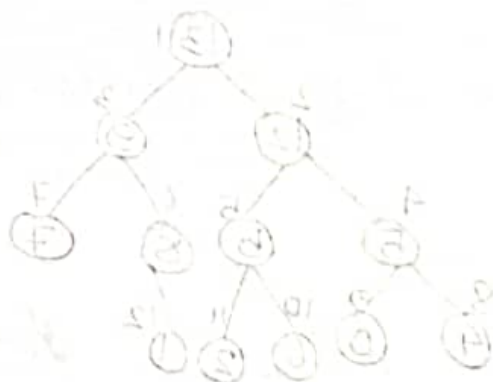
since $A[j] \leq x$, i will be incremented by 1 and then $A[i]$ and $A[j]$ will be exchanged.



This is the final configuration.

AVL and AVL will be swapped since AVL

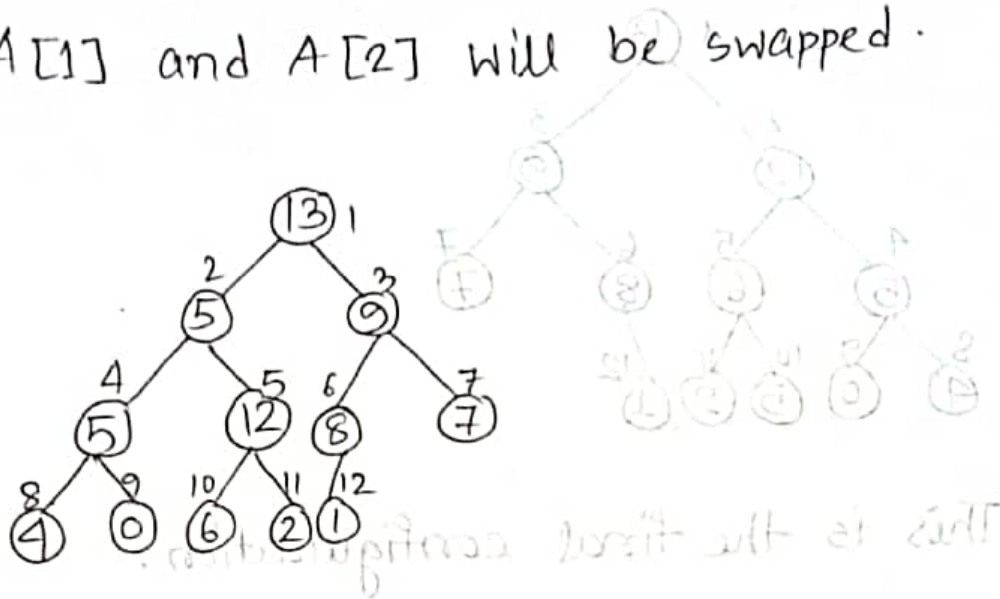
will restore the max heap property.



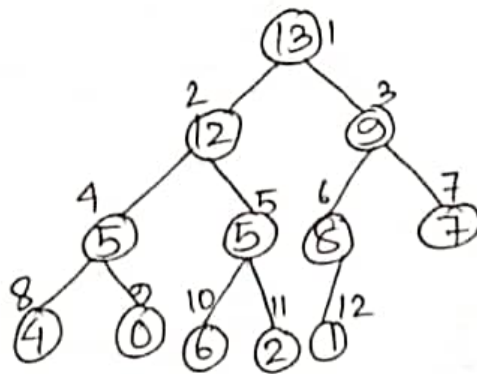
AVL and AVL will be swapped

$i = 1$

$A[1]$ and $A[2]$ will be swapped.



$A[2]$ and $A[5]$ will be swapped since $A[2]$ violates the max-heap property.



$A[5]$ and $A[10]$ will be swapped.

Here $A.\text{heap-size} = A.\text{length} = 12$. The non-leaf nodes starts from 1 to $\lceil \frac{12}{2} \rceil = 6$, we will call $\text{MAX-HEAPIFY}(A, i)$ for $i=6$ down to 1.

$i=6$

$A[6]$ is the largest element of the its sub-tree. No changes will be made.

$i=5$

$A[5]$ is the largest element of its sub-tree.

No changes will be made.

$i=4$

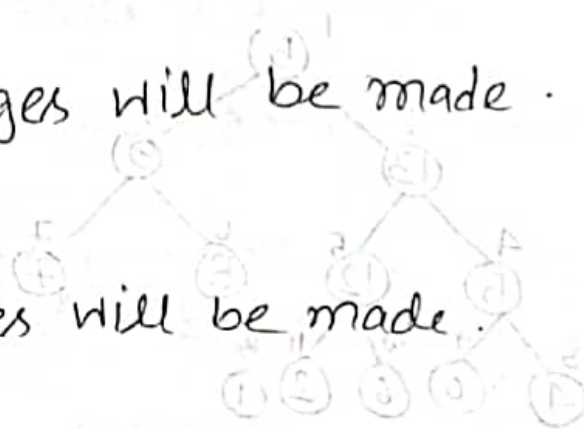
No changes will be made.

$i=3$

No changes will be made.

$i=2$

No changes will be made.



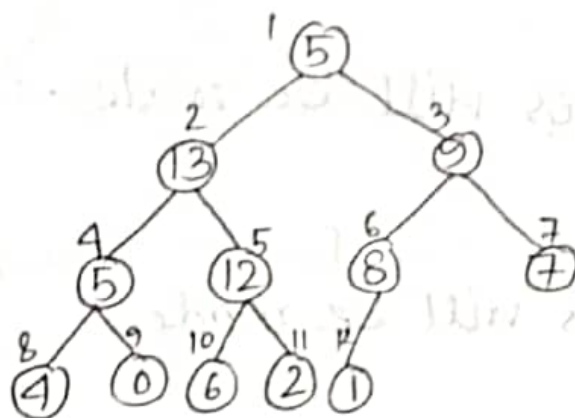
in its proper position since the elements on its left are less than or equal to it and it is less than or equal to the elements on its right side.

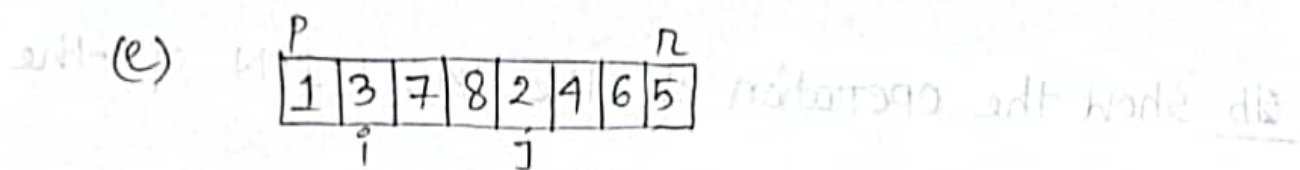
The call will return $(i+1)$ i.e. 5 as the index of pivot is 5 now.

2c. Illustrate the operation of Build-Max-Heap $(A, 1)$ on the following array.

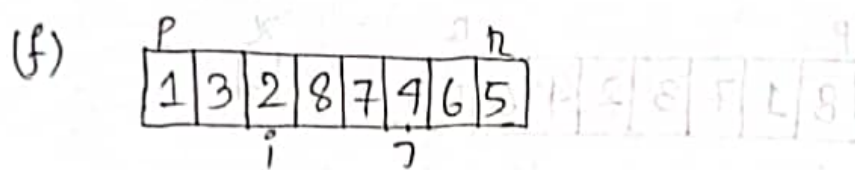
$A = \langle 5, 13, 9, 5, 12, 8, 7, 4, 0, 6, 2, 1 \rangle$

Ans:

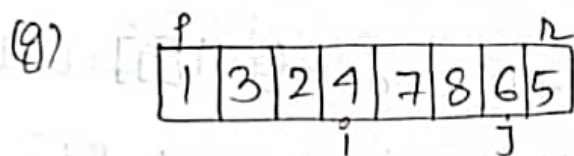




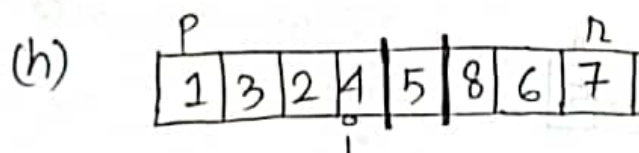
since $A[j] \leq x$, i will be incremented by 1 and then $A[i]$ and $A[j]$ will be exchanged.



Since $A[j] \leq x$, i will be incremented by 1 and then $A[i]$ and $A[j]$ will be exchanged.



now j will be incremented and the loop will be terminated since $j == R$. Then $A[i+1]$ and $A[R]$ will be exchanged.



The partition operation on the given array is done. Pivot element i.e. $x=5$ is placed

Ans. to Question no 3

3. What do you understand by optimization problem?

Which of the following problems are optimization problem? Give justification in support of your answer.

- i) Finding an empty class room from the list of class rooms.
- ii) Finding a way to best utilize the class room of the department

Ans: • Optimization problem :

Those problems are called optimization problems which can have many possible solution having a value and someone wish to find a solution with the optimal (minimum or maximum) value.

• Both problems are optimization problem.

Justification in support of my answer:

ii) It's an optimization problem. There exists more than one possible solution.

i) It's an optimization problem. More than one solution is possible.

Q What is overlapping Subproblem property? Show that following function for generating fibonacci sequence has this property.

Fibonacci (n)

if ($n \leq 1$) return n;

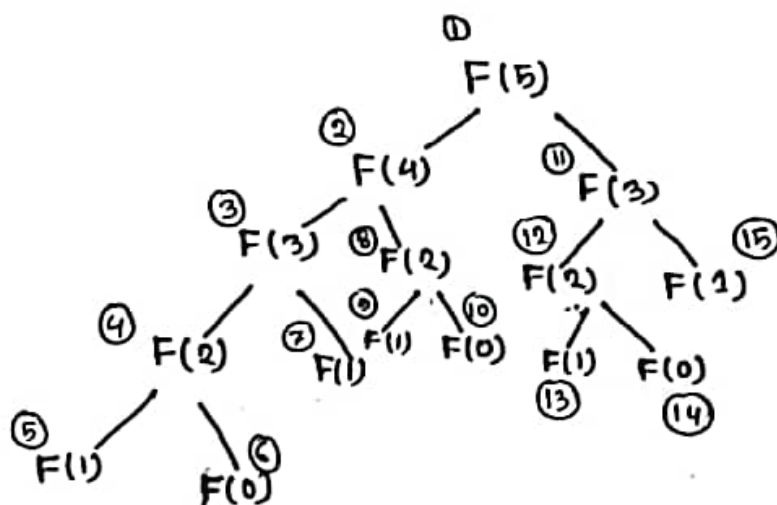
else return Fibonacci (n-1) + Fibonacci (n-2);

How can you solve the problem of finding fibonacci number shown above using dynamic programming?

Ans. • Overlapping Sub-problem property:

When a recursive algorithm revisits the same problem repeatedly, it is said that the optimization problem has overlapping sub-problem.

• For $n=5$, the recursive calling tree be like below:



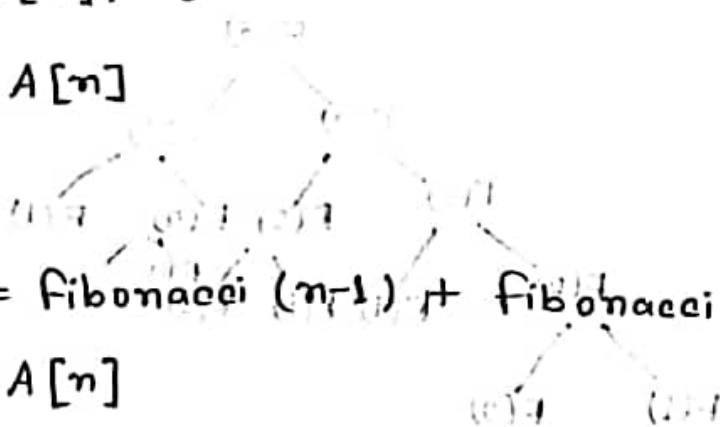
We can notice that $F(2)$ is called thrice
(function call numbers: 4, 8, 12). Hence according
to definition and the recursive tree above,
the given problem has overlapping sub-problem
property.

DP Solⁿ:

Let $A[1 \dots n]$ be a global array and is set 0 to
all the element initially.

Fibonacci (n):

1. if $n \leq 1$
2. return n
3. else if $A[n] \neq 0$
4. return $A[n]$
5. else
6. $A[n] = \text{Fibonacci}(n-1) + \text{Fibonacci}(n-2)$
7. return $A[n]$

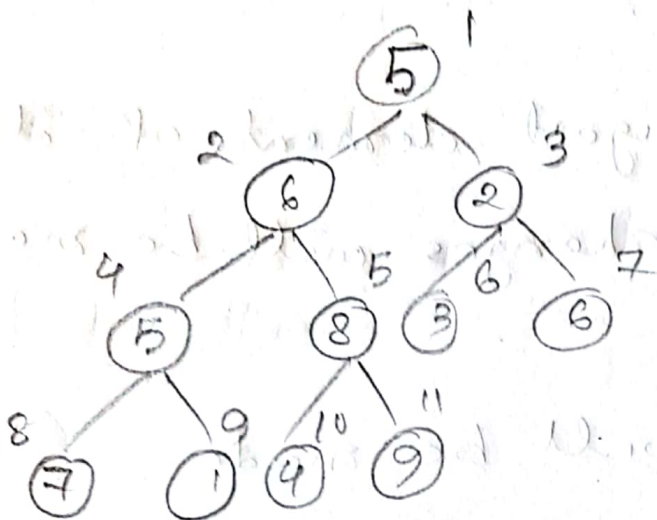


4

ID: C211262

a Illustrating the Build-Max-Heap(1)
on the following array:

$A = \{5, 6, 2, 5, 8, 3, 6, 7, 1, 4, 9\}$



Here A heap size $= A.length = 11$

The non-leaf nodes starts from 1 to

$11/2 = 5.5$. We will call MAX-HEAPIFY

(A, i) for $i = 5.5$ down to 1.

$$i = 5.5$$

$A[5.5]$ is the largest element of its subtree. No changes will be made.

$$i = 4.5$$

$A[4.5]$ is the largest element of its subtree. No changes will be made.

$$i = 3.5$$

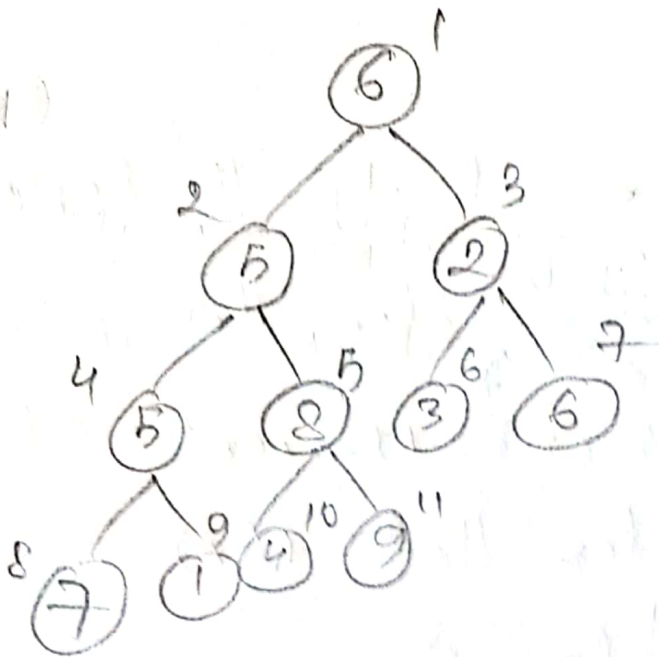
No changes will be made

$$i = 2.5$$

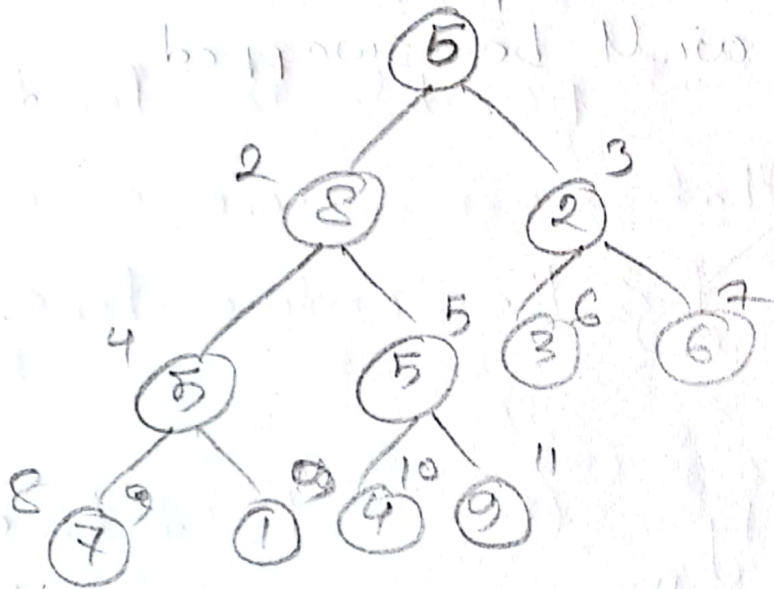
No changes will be made

$$i = 1.5$$

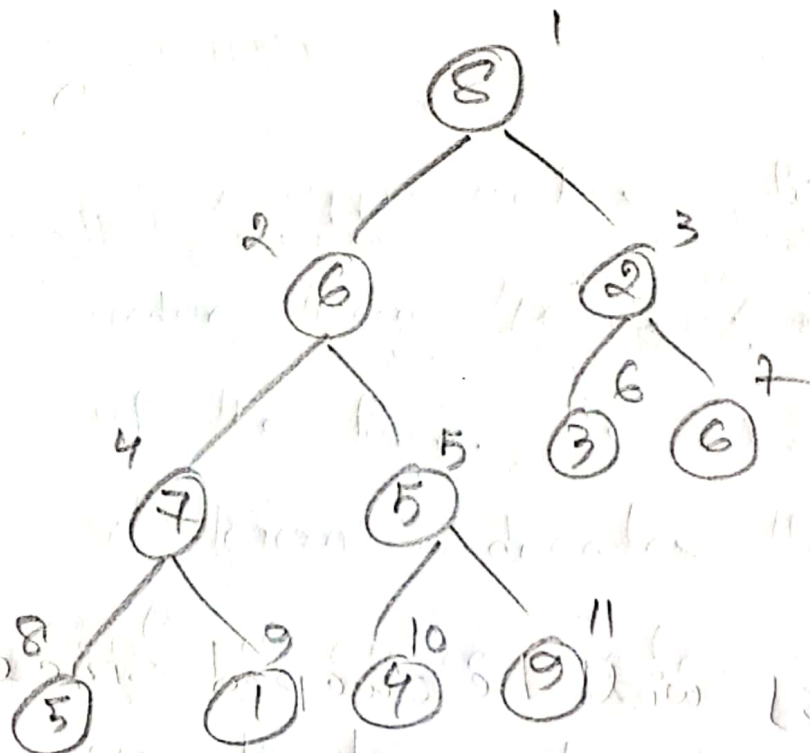
$A[1.5]$ & $A[2.5]$ will be swapped.



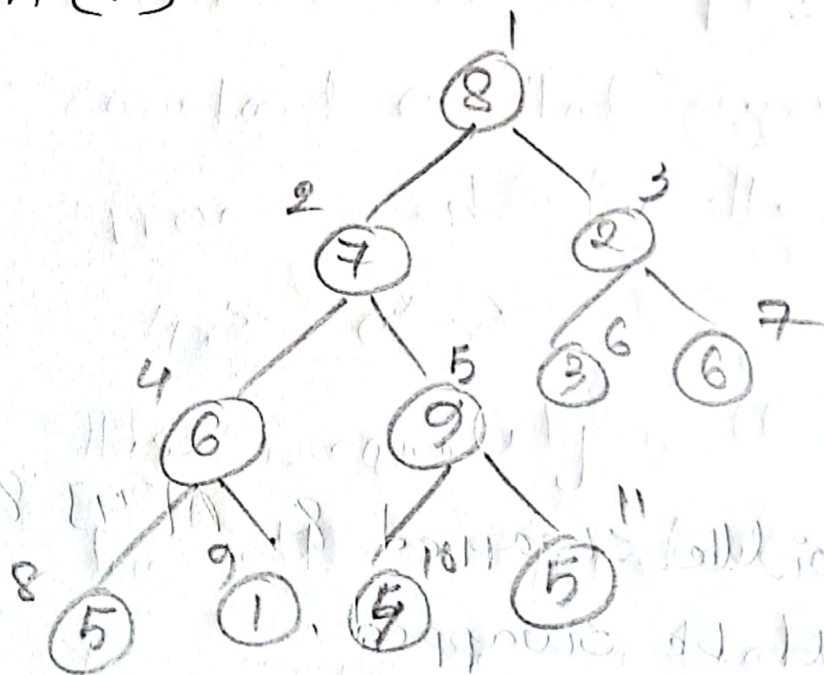
$A[2.5]$ & $A[5.5]$ will be swapped since $A[2.5]$ violates the max heap property.



$A[1.5]$ & $A[2]$ will be swapped & $A[4]$ & $A[8]$ will be swapped.

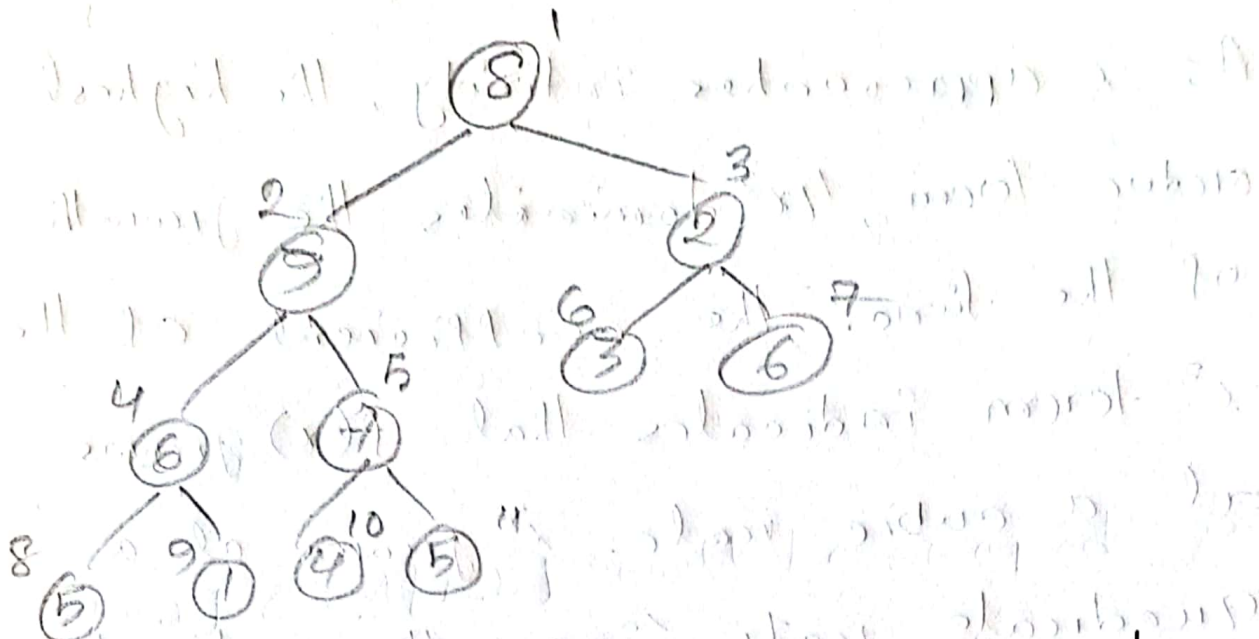


$A[4]$ & $A[2]$ will be swapped and
 $A[11]$ & $A[5]$ will be swapped

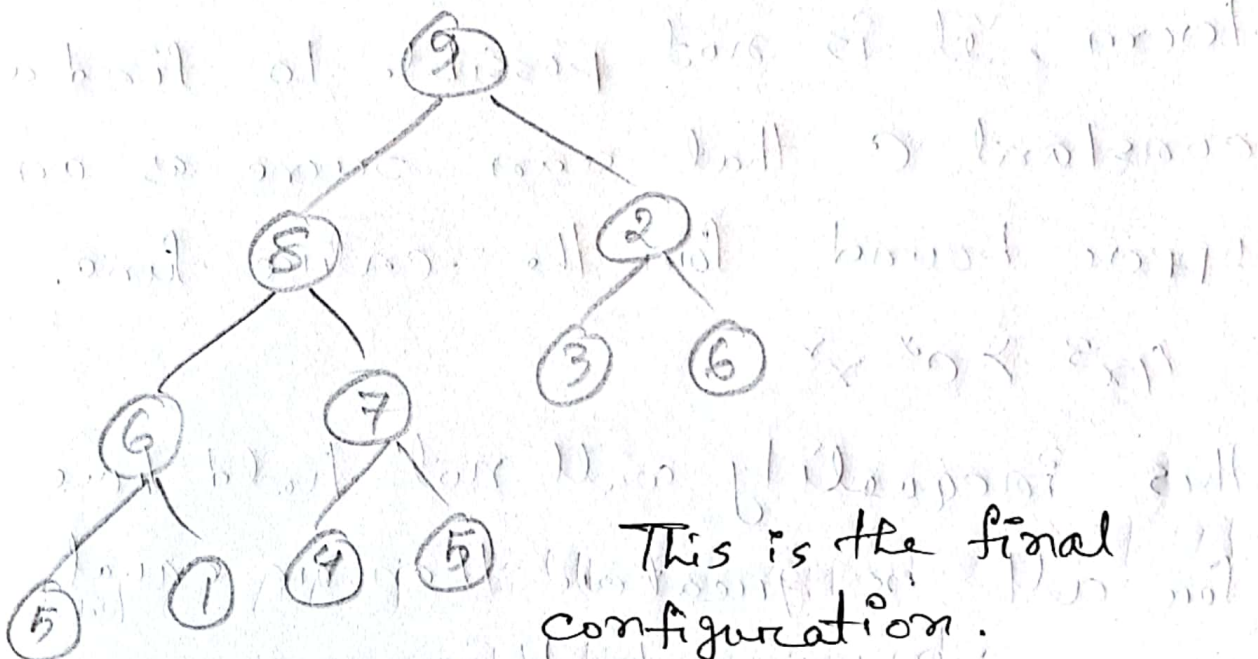


$A[5]$ & $A[2]$ will be swapped

Date



$A[1]$ & $A[2]$ will be swapped.



This is the final configuration.

Ans NO: 4b

A priority queue must at least support the following operations:

is_empty: Check whether the queue has no elements.

insert_with_priority: add an element to the queue with an associated priority.

pull_highest_priority_element: remove the element from the queue that has the highest priority, and return it.

Q211202

C

As x approaches infinity, the highest order term, $4x^3$ dominates the growth of the func. The coefficient of the x^3 term indicates that $f(x)$ grows at a cubic rate. x^r grows at a quadratic rate. Since the cubic term grows faster than the quadratic term, it is not possible to find a constant C that can serve as an upper bound for the entire func.

$$4x^3 > C^* x^r$$

This inequality will not hold true for all x greater than or equal to some x , which means $f(x) = 4x^3 - 5x + 3$ is not $O(x^r)$

Ans No 9d

Four steps to develop a dynamic programming algorithm:

1. Characterize the structure of an optimal solution.
2. Recursively define the value of an optimal solution.
3. Compute the value of an optimal solution, typically in a bottom-up fashion.
4. Construct an optimal solution from the computed information.